SWIFTDATA · PRACTICE PROJECT

PantryPal

A kitchen inventory & recipe tracker to verify your SwiftData learning.

Based on "SwiftData by Example" (hackingwithswift.com) · Sections 1–5

> **Goal:** Build a single-user iOS app that tracks what's in your kitchen and the recipes you can make from it. It's designed to force every relationship type, filtering, sorting, uniqueness, enums, and multi-store config — without being another to-do clone. Build it, break it, then come back for the quiz.

What you're building

PantryPal stores **ingredients** you own and **recipes** that use them. It tells you what you can cook right now based on current stock, warns you about items nearing expiry, and lets you organize recipes into categories.

The data model

This is where most of your learning gets verified. Implement these models with the exact SwiftData features noted.

`@Model` Ingredient

- `name: String` — make it a **unique attribute** (`#Unique`) so "Flour" can't be added twice
- `quantity: Double`
- `unit: Unit` — an **enum** (`.grams` , `.ml` , `.pieces`) stored as a property
- `location: StorageLocation` — enum (`.fridge` , `.freezer` , `.pantry`)
- `addedDate: Date` , `expiryDate: Date?`
- `daysUntilExpiry: Int` — a **derived / computed attribute** from `expiryDate`
- Use a **transient attribute** somewhere (e.g. a computed display string that isn't persisted)

`@Model` Recipe

- `title: String` , `instructions: String` , `servings: Int`
- `difficulty: Difficulty` — enum
- `isFavorite: Bool`

`@Model` RecipeIngredient — the interesting one

A join model linking a `Recipe` to an `Ingredient` with an `amountNeeded: Double`. This is your **many-to-many-with-payload** exercise.

`@Model` Category & NutritionInfo

- `Category` — a label like "Baking" or "Breakfast"
- `NutritionInfo` — optional per-recipe detail (for the one-to-one exercise)

Relationships you must implement

TYPE	WHERE
One-to-many	`Category` → many `Recipe`s
One-to-one	each `Recipe` has one optional `NutritionInfo`
Many-to-many	`Recipe` ↔ `Ingredient` (via `RecipeIngredient`)
Cascade delete	deleting a Recipe removes its join rows + NutritionInfo, but *not* the shared Ingredients
Inferred vs explicit	use an inferred relationship somewhere and an explicit `@Relationship` elsewhere
Min/max constraint	a recipe needs at least 1 ingredient

Features to implement

1. **List view with `@Query`** showing all ingredients.
2. **Sorting** — toggle between sort by name, expiry date, and quantity (`SortDescriptor` + dynamic sort).
3. **Filtering with predicates** — segmented control for Fridge / Freezer / Pantry, plus a search field by name. Exercises **dynamically changing a query's predicate**.
4. **Create / edit / delete** — add form, edit via `@Bindable`, swipe-to-delete. Editing verifies `@Bindable` vs `@Binding`.
5. **Recipe detail** showing its ingredients (relationship traversal) and whether you have enough of each in stock.
6. **"What can I cook?" view** — only recipes whose ingredients are all in stock. A predicate + relationship stress test.
7. **Animate** query changes when items are added or removed.

Configuration checklist

- Set up the `ModelContainer` in your `App` struct via `.modelContainer(for:)`.
- Add a **second configuration** — e.g. a separate in-memory store for preview mode, or an on-disk store with a **custom filename**.
- Show you understand **autosave**: leave it on normally, but add one flow (a multi-step "add recipe" wizard) where you **disable autosave** and save manually on Done.
- Use **SwiftData in previews** with an in-memory container seeded with sample data.

Self-verification checklist

You've learned it well if you can answer "yes" to all of these:

- [ ] Adding a duplicate ingredient name is rejected (unique constraint works)
- [ ] Deleting a recipe removes its join rows + nutrition but leaves ingredients intact (cascade)
- [ ] The list re-sorts live when I switch sort mode (dynamic sort)
- [ ] Search + location filter combine correctly (dynamic predicate)
- [ ] Editing an ingredient in a detail view updates the list without manual refresh (`@Bindable`)
- [ ] Previews render with fake data and never touch my real store (in-memory config)
- [ ] I can explain out loud the difference between `ModelContainer`, `ModelContext`, and `ModelConfiguration`

Stretch goals optional — later sections

These pull from parts of the guide beyond sections 1–5 (Working with data, Migration, Architecture, Solving problems). Skip if you only studied the first five sections.

- **Pre-load** the app with a JSON file of starter ingredients on first launch.
- Add **undo/redo** support for delete actions.
- Refactor one screen to **MVVM** to separate SwiftData from the view.
- Do a **lightweight migration** by adding a new property (e.g. `barcode`) in a v2 schema.
- Write one **unit test** that inserts, fetches, and asserts on an in-memory container.
